



TEST LOGICIEL

DÉCOUVERTE



TEST LOGICIEL

SOMMAIRE

- Introduction
- Les types de tests
- Les méthodes de tests
- Les tests en entreprise
- Automatisation des tests
- Aller plus loin



INTRODUCTION

CRÉER UNE APPLICATION

LE DÉVELOPPEMENT D'UNE APPLICATION N'A PAS DE FIN

Évolution des besoins

Les besoins des utilisateurs et du marché peuvent évoluer avec le temps. Les nouvelles fonctionnalités, les améliorations de performance et les ajustements doivent être continuellement pris en compte pour répondre aux attentes changeantes.

Technologies en évolution

Les technologies utilisées pour le développement d'applications évoluent constamment. Les mises à jour de langages de programmation, de frameworks et de bibliothèques peuvent nécessiter des ajustements continus pour maintenir la compatibilité et tirer parti des nouvelles fonctionnalités.

Correctifs et mises à jour

Les bugs et les problèmes de sécurité peuvent survenir après le lancement de l'application. Les développeurs doivent fournir des correctifs et des mises à jour régulières pour garantir la stabilité et la sécurité de l'application.

Avis des utilisateurs

Les retours des utilisateurs sont précieux pour améliorer l'expérience utilisateur. L'intégration de nouvelles fonctionnalités basées sur les commentaires des utilisateurs peut être un processus continu.

LES EFFETS DE BORD

SIDE EFFECTS EN ANGLAIS



Les effets de bord, également connus sous le nom de "side effects" en anglais, se réfèrent aux changements observables à l'extérieur d'une fonction ou d'une portion de code lorsqu'elle est exécutée. Les effets de bord peuvent avoir des conséquences non intentionnelles sur le comportement global d'une application.

Pour minimiser les effets de bord et rendre le code plus prévisible, il est souvent recommandé d'adopter des principes de programmation fonctionnelle, où les fonctions n'ont pas d'effets de bord et où les données sont immuables autant que possible. Cela favorise la lisibilité, la testabilité et la maintenance du code.

Connaître les subtilités d'un langage fera de vous des meilleurs développeurs,
répondons ensemble à la question qui suit

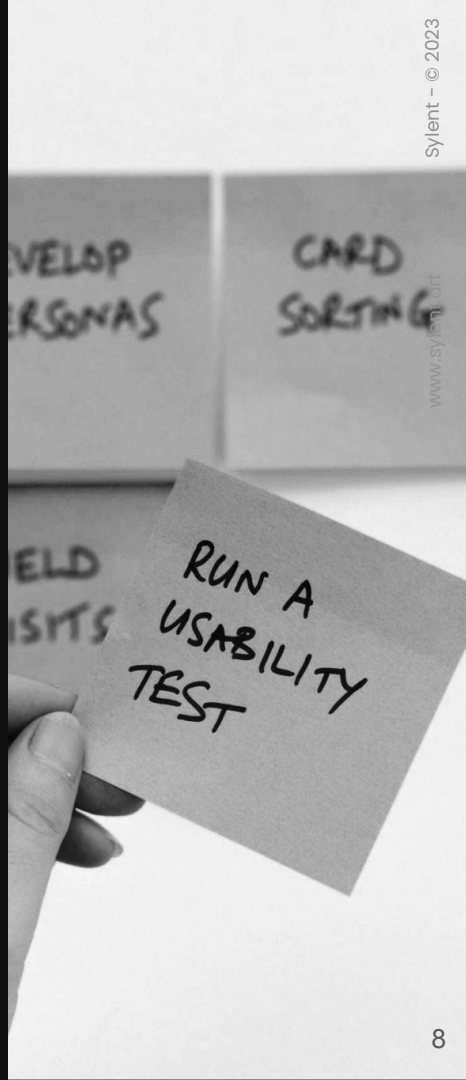
JS

let vs var ?

PROBLÉMATIQUE

COMMENT GARANTIR LE BON FONCTIONNEMENT D'UNE APPLICATION ?

Les tests logiciels permettent de s'assurer du bon fonctionnement d'une application. Il existe plusieurs types de tests et d'approches que nous allons voir et détailler dans la suite du cours.



LA COUVERTURE FONCTIONNELLE

INDICATEUR PRATIQUE

Il existe un indicateur permettant de garantir le bon fonctionnement d'une application, il s'agit de la couverture fonctionnelle. Celle-ci est définie par le ratio des fonctionnalités testées sur l'ensemble des fonctionnalités.

En général, pour garantir la qualité d'une application, il est recommandé d'avoir une couverture fonctionnelle excédant les 80%.

$$\text{couverture fonctionnelle} = \frac{\text{fonctionnalités testées}}{\text{ensemble des fonctionnalités}}$$



LE TEST LOGICIEL AVEC FIRESHIP

100 SECONDS OF





LES TYPES DE TESTS

TESTS UNITAIRES

TESTER UNE FONCTION OU UNE MÉTHODE

Un test unitaire se concentre sur une seule unité, qui est le plus petit élément identifiable de notre application. Selon les langages de programmation, plusieurs éléments du code peuvent constituer une unité. Il peut s'agir d'une fonction, d'une méthode de classe, d'un module ou d'un objet.

Vous avez ci-contre un exemple de test unitaire vérifiant le bon fonctionnement de la fonction "sum".

```
export function sum(a, b) {  
  return a + b  
}
```

sum.js

```
import { expect, test } from 'vitest'  
import { sum } from '../js/sum'  
  
test('adds 1 + 2 to equal 3', () => {  
  expect(sum(1, 2)).toBe(3)  
})
```

sum.test.js



LE COÛT D'UN TEST UNITAIRE

NE JAMAIS OUBLIER QUE TOUT A UN COÛT

Code supplémentaire à écrire

Maintenir dès que le code change

Doit être lancés régulièrement

Peut provoquer des faux positifs ou faux négatifs

```
export function sum(a, b) {  
  return a + b  
}
```

sum.js

```
import { expect, test } from 'vitest'  
import { sum } from '../js/sum'  
  
test('adds 1 + 2 to equal 3', () => {  
  expect(sum(1, 2)).toBe(3)  
})
```

sum.test.js



LES CARACTÉRISTIQUES D'UN BON TEST UNITAIRE

NE JAMAIS OUBLIER QUE TOUT A UN COÛT

Facile à lire	Prévisible et déterministe
Automatique et répétable	Fonctionne entièrement en RAM
Exécutable en un clic / commande	Synchrone et séquentiel
Très rapide (en secondes)	Entièrement isolé des autres tests (parallélisable)

TESTS D'INTÉGRATION

TESTER PLUSIEURS COMPOSANTS ET SERVICES

Un test d'intégration vérifie le bon fonctionnement des interactions entre plusieurs composants (Base de données, API...). Ces composants sont aussi appelés des dépendances volatiles.

L'objectif principal est de s'assurer que les différentes parties d'une application fonctionnent correctement lorsqu'elles sont intégrées.

Dépendances volatiles

Introduit un pré-requis (BDD, API) ou non déterministe (date, UUID)...

Dépendance stables

Toujours disponible, ne nécessite aucun setup/cleanup et est rapide d'exécution.

```
function feature() {  
  const data = useDatabase('dsn')  
  return <main>{data}</main>  
}
```


TESTS END-TO-END

AUSSI APPELÉS TESTS FONCTIONNELS

Les tests E2E sont une catégorie de tests logiciels qui simulent le comportement d'un utilisateur final interagissant avec une application dans son ensemble.

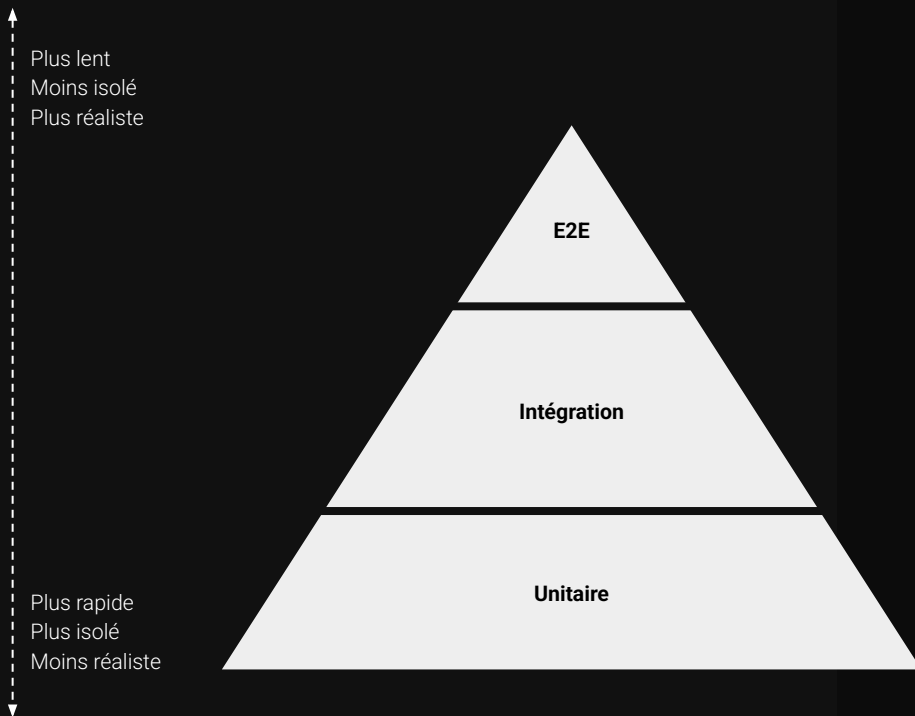
Le but des tests E2E est de vérifier que toutes les composantes d'une application fonctionnent correctement ensemble, de bout en bout, en reproduisant des scénarios d'utilisation réalistes.

```
describe('login', () => {  
  it('user should be able to log in', () => {  
    cy.visit('/')  
  
    // Open the login modal  
    cy.get('button').contains('Login').click()  
  
    // Fill in the form  
    cy.get('input[type="email"]').type('test@test.com')  
    cy.get('input[type="password"]').type('test123')  
  
    // Submit the form  
    cy.get('button').contains('Sign in').click()  
    cy.contains('button', 'Logout').should('be.visible')  
  })  
})
```



SPÉCIFICITÉS DE CHACUN DES TYPES DE TESTS

LA PYRAMIDE DES CARACTÉRISTIQUES



Les tests unitaires constituent la base de la pyramide. Ils sont nombreux, rapides et peu coûteux à exécuter, et leur débogage est facile. Mais la confiance qu'ils vous donnent dans le fait que l'application fonctionne réellement comme prévu est minime.

Tandis que les tests E2E au sommet offre la plus grande confiance, mais moins de couverture car les cas à tester deviennent innombrables entraînant un coût de maintenance très élevé.



LES MÉTHODES DE TESTS

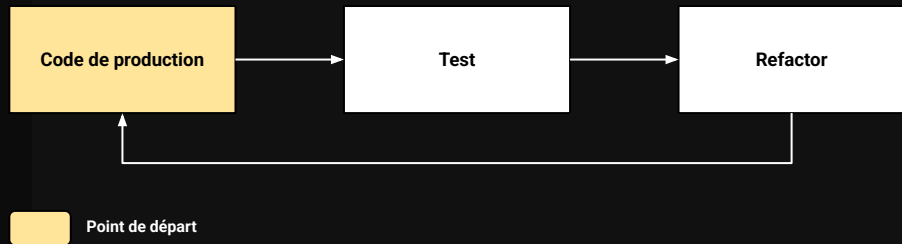
LE TEST LAST

LORSQUE QUE L'ON PART D'UN EXISTANT

Écriture des tests après avoir écrit le code de production

Pas recommandé : rend les tests plus difficiles à écrire

Utile sur les systèmes legacy



LE TEST FIRST

PRÉVOIR LES DIFFÉRENTS SCÉNARIOS EN AMONT

Écriture des tests avant avoir écrit le code de production

Spécifie en amont les attentes du client

Tests d'acceptation / Tests d'intégration

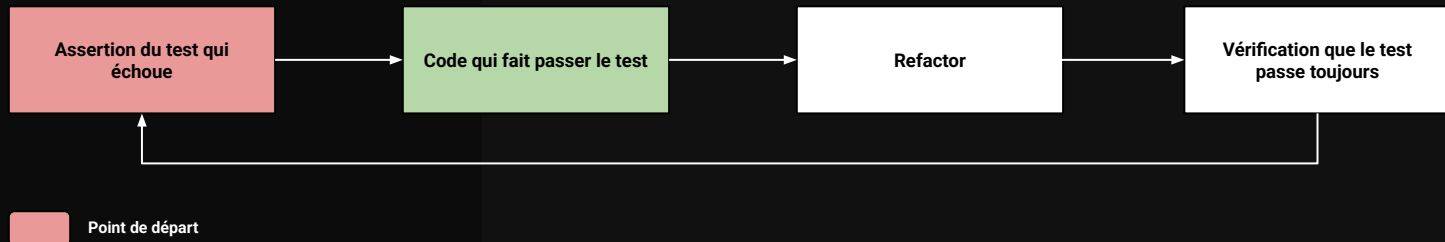


LE TEST DRIVEN DEVELOPMENT

ACRONYME : TDD - DÉVELOPPEMENT PILOTÉ PAR LES TESTS

Développement itératif conduit par les tests

Aide à faire émerger une solution et à améliorer le design





LES TESTS EN ENTREPRISE

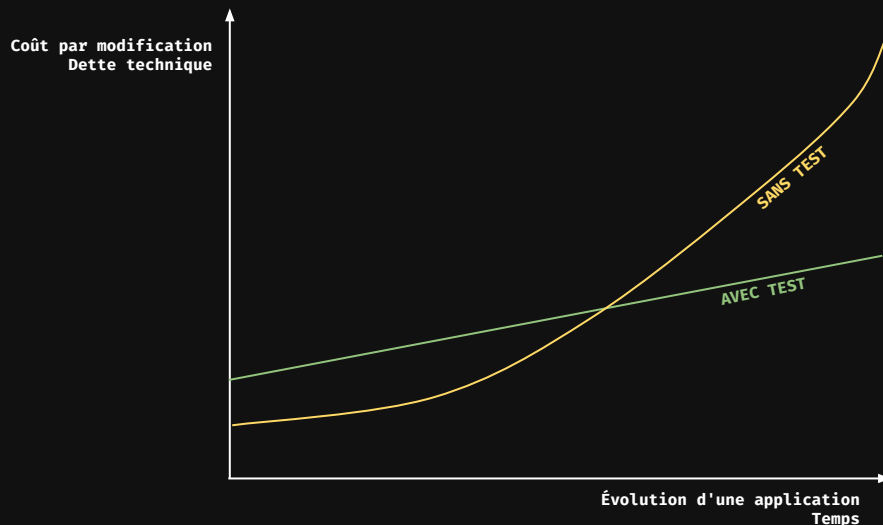
LA DETTE TECHNIQUE

CAS D'UNE ENTREPRISE DÉPOURVUE DE TESTS LOGICIELS

La dette technique fait référence à l'accumulation de compromis et de raccourcis pris lors du développement logiciel qui peuvent entraîner des conséquences négatives à long terme.

La dette technique se manifeste lorsque des choix sont faits pour livrer rapidement un produit ou une fonctionnalité sans prendre le temps nécessaire pour mettre en place des solutions robustes, maintenables et extensibles. Ces choix peuvent inclure des pratiques de codage de qualité inférieure, l'omission de tests appropriés, l'utilisation de bibliothèques obsolètes, ou encore des décisions architecturales discutables.

Comme toute dette, la dette technique doit être remboursée à un moment donné. Si elle n'est pas traitée, elle peut entraîner des problèmes tels que des retards dans le développement, des bogues fréquents, des coûts de maintenance élevés et une réduction de la productivité de l'équipe de développement.



ENJEUX DES TESTS

LES AVANTAGES SONT NOMBREUX

Meilleure qualité de vie de développeurs

Réduction des coûts de développement sur le long terme

Augmentation de la confiance des utilisateurs finaux de la solution

Argument commercial



DEVONS-NOUS TOUT TESTER ?

CE N'EST SOUVENT PAS NÉCESSAIRE

Développer une application requiert d'être pragmatique : les efforts déployés doivent être en cohérence avec la valeur produite.

Il existe des cas où écrire des tests a peu de sens, en voici quelques exemples :

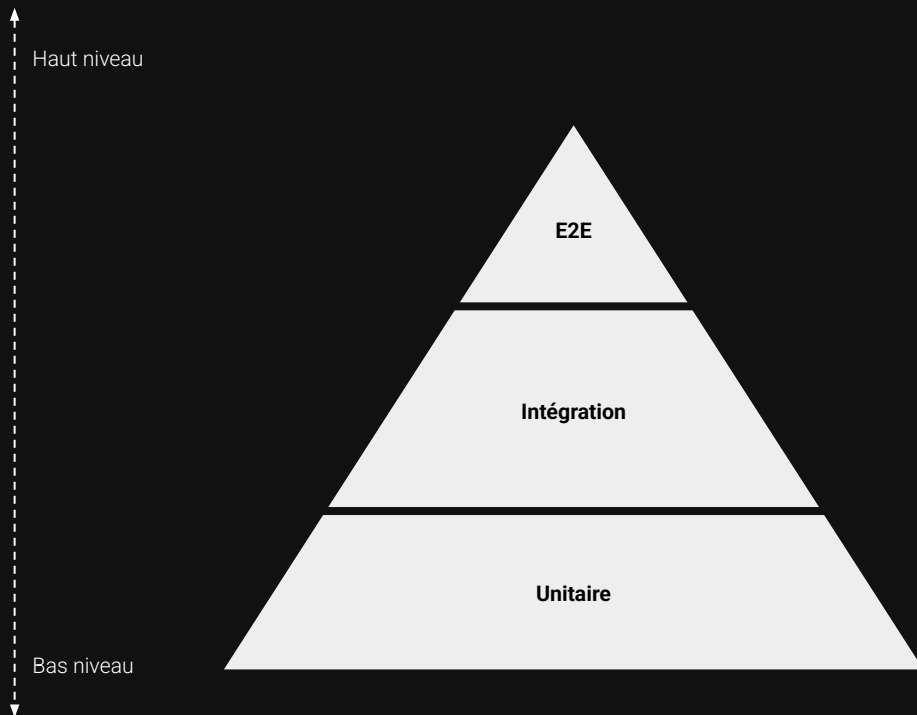
Lorsque des éléments d'un framework ou d'un langage rendent les tests très coûteux à mettre en place. Ce coût peut provenir de l'effort nécessaire pour pouvoir écrire le test ou du temps d'exécution des tests.

Lorsque le code que l'on souhaite tester a été généré, cela peut l'être par un IDE ou par autre outil (IA). Les POJO (Plain Old Java Object) par exemple sont des classes pleines de code boilerplate (getters, setters, equal...) qui sont facile à générer par un IDE (Eclipse).



HAUT NIVEAU ET BAS NIVEAU

NOTION



CAS D'ENTREPRISE

SCÉNARIO

L'entreprise Factufacile est une entreprise qui a développé une application web permettant de gérer la comptabilité et les factures de ses clients.

Cette application web a été réalisée à l'aide d'un **framework PHP maison**. Elle comporte un **système d'authentification** et **diverses fonctionnalités concernant la gestion ainsi que la création de factures**.

Pour le moment cette application ne comprend aucun test, quel est l'ordre de priorité des éléments à tester ?



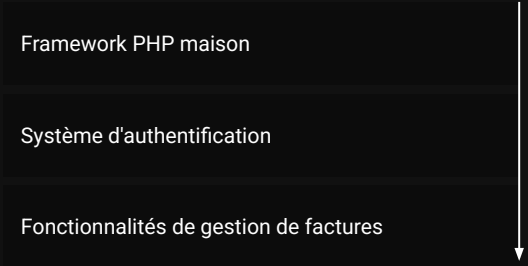
CAS D'ENTREPRISE

STRATÉGIE DE TEST LOGICIEL

STRATÉGIE TOP-DOWN



STRATÉGIE BOTTOM-UP

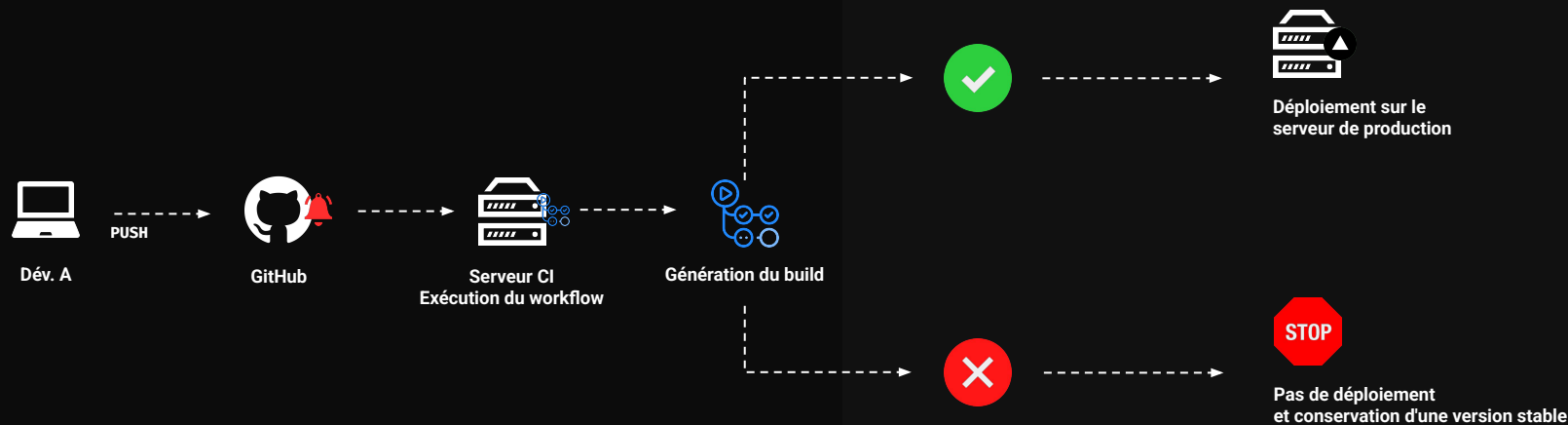




AUTOMATISATION DES TESTS

INTÉGRATION ET DÉPLOIEMENT CONTINU

AUTOMATISATION - ANG. : PIPELINE CI/CD





CI/CD AVEC FIRESHIP

100 seconds of



CI

CD

DEVOPS

EXPLAINED





ALLER PLUS LOIN

DÉCOUVERTE DES FRAMEWORKS DE TEST

ILS SONT NOMBREUX

xUnit
PHPUnit
JUnit
Atoum
Cypress
Jest
Pytest
PEST
Vitest
Playwright

DÉCOUVERTE DES OUTILS DE CI

AUTOMATISER L'EXÉCUTION DES TESTS

Jenkins

Outil très populaire d'intégration continue de type CI/CD écrit en Java, Jenkins est une application open source conçue pour orchestrer des pipelines de déploiement. Dotée d'une API, elle propose pas moins de 1500 plugins.

CircleCI

CircleCI est un outil d'intégration continue qui automatise le processus de développement, de test et de déploiement des applications. Elle permet aux développeurs de détecter rapidement les erreurs, d'assurer la qualité du code et d'accélérer le cycle de développement. CircleCI est simple à configurer et favorise la livraison continue d'un logiciel.

GitHub Actions

GitHub Actions est un outil d'intégration et de déploiement continue. Étant directement intégré à la plateforme GitHub et se configurant à l'aide d'un fichier de configuration YAML, il est très simple à mettre en place.



www.sylvain-art.com - © 2024



DÉCOUVERTE DES OUTILS DE CD

DÉPLOYER SI TOUS LES TESTS PASSENT

Vercel

Vercel est une plateforme cloud qui permet aux développeurs de déployer rapidement et efficacement des applications web. Il simplifie le processus de développement grâce à des déploiements automatiques via l'intégration Git.

Render

Render est une plateforme cloud moderne qui simplifie le déploiement d'applications et de sites web, en intégrant des fonctionnalités de CI/CD. Il permet des déploiements automatiques à partir de repositories GitHub.



www.sylvain-art.com - © 2024



ALLER PLUS LOIN



MERCI DE VOTRE ATTENTION